

Data Visualization App

Aim

Design and develop a mobile app that displays user data (e.g., steps, expenses) using custom-designed charts and graphs. Design Focus: Data visualization, clarity.

Procedure

1. Requirement Analysis

Identify the objective of developing a mobile application that displays user financial data such as **daily expenses** using multiple charts and graphs. The aim is to represent the data clearly through visual components such as **bar chart, line chart, and pie chart**.

2. Create an Android Studio Project

- Open **Android Studio**.
- Select **New Project** → **Empty Activity**.
- Enter the application name as **Data Visualization App**.
- Choose **Kotlin** as the programming language.
- Select **Minimum SDK (API 21 or above)**.

3. Add Required Library Dependency

- Open the **build.gradle (Module: app)** file.
- Add the dependency for **MPAndroidChart** library.
- Sync the project to download and configure the chart library.

4. Design the User Interface

- Open the layout file `activity_main.xml`.
- Use a **ScrollView** as the root layout to support vertical scrolling.
- Add a **ConstraintLayout** inside the **ScrollView**.
- Place a **TextView** for the dashboard title.
- Add summary **TextViews** to display:
 - Total Expense
 - Highest Spending Day
- Add a **Spinner** to select the day of the week.
- Add a **TextInputEditText** to enter the expense amount.
- Add a **Button** to update the dashboard.
- Add **BarChart, LineChart, and PieChart** components to visualize the expense data.

5. Initialize the Dataset

- In the `MainActivity.kt` file, create a list of day labels such as **Mon, Tue, Wed, Thu, Fri, Sat, Sun**.

- Initialize a mutable list of sample expense values for the seven days.
- Use these sample values to display charts immediately when the app starts.
- 6. **Configure the Spinner**
 - Create an **ArrayAdapter** using the list of day labels.
 - Set the adapter to the Spinner so that the user can choose a day for updating the expense value.
- 7. **Configure the Chart Components**
 - Initialize the **BarChart**, **LineChart**, and **PieChart** in the activity.
 - Disable unnecessary chart descriptions.
 - Configure X-axis labels using the list of days.
 - Disable the right Y-axis where required.
 - Set minimum axis values and enable chart animations for better presentation.
- 8. **Implement Data Update Logic**
 - Read the amount entered by the user from the input field.
 - Validate the input to ensure:
 - the field is not empty
 - the entered value is numeric
 - the amount is not negative
 - Get the selected day from the Spinner.
 - Update the corresponding expense value in the dataset.
- 9. **Populate the Bar Chart**
 - Convert the expense values into **BarEntry** objects.
 - Create a **BarDataSet** and apply suitable colors and value labels.
 - Set the dataset to the **BarChart** using **BarData**.
 - Refresh the chart using `invalidate()`.
- 10. **Populate the Line Chart**
 - Convert the expense values into **Entry** objects.
 - Create a **LineDataSet** to represent the weekly expense trend.
 - Customize the line chart with colors, circle markers, filled area, and value labels.
 - Set the dataset to the **LineChart** using **LineData**.
 - Refresh the chart using `invalidate()`.
- 11. **Populate the Pie Chart**
 - Convert non-zero expense values into **PieEntry** objects.
 - Create a **PieDataSet** to represent spending distribution.
 - Apply suitable colors, slice spacing, and value text size.
 - Set the dataset to the **PieChart** using **PieData**.
 - If no data is available, display **No Data Available** in the chart center.
 - Refresh the chart using `invalidate()`.
- 12. **Update Dashboard Summary**
 - Calculate the **total weekly expense** by summing all values in the dataset.
 - Find the **highest spending day** by identifying the maximum expense value.

- Display these details in the summary TextViews at the top of the dashboard.

13. Provide User Feedback

- Clear the input field after updating the data.
- Display a **Toast message** to confirm that the dashboard has been updated successfully.

14. Test the Application

- Run the application in the **Android Emulator** or on a physical device.
- Enter different expense values for various days.
- Verify that all three charts and summary details update correctly.

15. Build and Run the Application

- Clean and rebuild the project.
- Execute the application and confirm that the dashboard displays expense data clearly using charts and summary information.

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/Theme.DATAVISUALIZATIONAPP">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="@string/app_name"
            android:theme="@style/Theme.DATAVISUALIZATIONAPP">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true">

    <androidx.constraintlayout.widget.ConstraintLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="16dp">

        <TextView
            android:id="@+id/tvTitle"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Financial Dashboard"
            android:textColor="@android:color/black"
            android:textSize="24sp"
            android:textStyle="bold"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent" />

        <TextView
            android:id="@+id/tvSummary"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginTop="12dp"
            android:text="Total Expense: $0.00"
            android:textColor="@android:color/black"
            android:textSize="16sp"
            android:textStyle="bold"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@id/tvTitle" />

        <TextView
            android:id="@+id/tvHighest"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginTop="4dp"
            android:text="Highest Spending Day: -"
            android:textColor="@android:color/black"
            android:textSize="15sp"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@id/tvSummary" />

        <LinearLayout
            android:id="@+id/inputLayout"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginTop="16dp">
```

```

        android:orientation="horizontal"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/tvHighest">

        <Spinner
            android:id="@+id/spinnerDays"
            android:layout_width="0dp"
            android:layout_height="48dp"
            android:layout_weight="1" />

        <com.google.android.material.textfield.TextInputLayout
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginStart="8dp"
            android:layout_weight="1"
            android:hint="Amount ($)">

            <com.google.android.material.textfield.TextInputEditText
                android:id="@+id/etAmount"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:inputType="numberDecimal" />
        </com.google.android.material.textfield.TextInputLayout>
    </LinearLayout>

    <Button
        android:id="@+id/btnUpdate"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:text="Update All Charts"
        app:layout_constraintTop_toBottomOf="@id/inputLayout" />

    <TextView
        android:id="@+id/labelBar"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="24dp"
        android:text="1. Bar Chart: Daily Comparison"
        android:textStyle="bold"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/btnUpdate" />

    <com.github.mikephil.charting.charts.BarChart
        android:id="@+id/barChart"
        android:layout_width="match_parent"
        android:layout_height="250dp"
        android:layout_marginTop="8dp"
        app:layout_constraintTop_toBottomOf="@id/labelBar" />

    <TextView
        android:id="@+id/labelLine"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="24dp"

```

```

        android:text="2. Line Chart: Weekly Trend"
        android:textStyle="bold"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/barChart" />

<com.github.mikephil.charting.charts.LineChart
    android:id="@+id/lineChart"
    android:layout_width="match_parent"
    android:layout_height="250dp"
    android:layout_marginTop="8dp"
    app:layout_constraintTop_toBottomOf="@id/labelLine" />

<TextView
    android:id="@+id/labelPie"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="24dp"
    android:text="3. Pie Chart: Spending Distribution"
    android:textStyle="bold"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/lineChart" />

<com.github.mikephil.charting.charts.PieChart
    android:id="@+id/pieChart"
    android:layout_width="match_parent"
    android:layout_height="300dp"
    android:layout_marginTop="8dp"
    android:layout_marginBottom="16dp"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintTop_toBottomOf="@id/labelPie" />

</androidx.constraintlayout.widget.ConstraintLayout>
</ScrollView>

```

MainActivity.kt

```

package com.example.datavisualizationapp

import android.graphics.Color
import android.os.Bundle
import android.widget.ArrayAdapter
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import com.example.datavisualizationapp.databinding.ActivityMainBinding
import com.github.mikephil.charting.components.Legend
import com.github.mikephil.charting.components.XAxis
import com.github.mikephil.charting.data.BarData
import com.github.mikephil.charting.data.BarDataSet
import com.github.mikephil.charting.data.BarEntry
import com.github.mikephil.charting.data.Entry
import com.github.mikephil.charting.data.LineData
import com.github.mikephil.charting.data.LineDataSet
import com.github.mikephil.charting.data.PieData
import com.github.mikephil.charting.data.PieDataSet
import com.github.mikephil.charting.data.PieEntry

```

```

import com.github.mikephil.charting.formatter.IndexAxisValueFormatter
import com.github.mikephil.charting.formatter.ValueFormatter
import com.github.mikephil.charting.utils.ColorTemplate
import java.util.Locale

class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding

    private val labels = listOf("Mon", "Tue", "Wed", "Thu", "Fri", "Sat",
"Sun")

    // Default sample data for immediate chart display
    private val expenses = mutableListOf(120f, 180f, 90f, 250f, 300f, 140f,
200f)

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        setupSpinner()
        setupCharts()
        updateSummary()

        binding.btnUpdate.setOnClickListener {
            updateData()
        }
    }

    private fun setupSpinner() {
        val adapter = ArrayAdapter(this,
android.R.layout.simple_spinner_item, labels)

        adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item
)

        binding.spinnerDays.adapter = adapter
    }

    private fun updateData() {
        val amountStr = binding.etAmount.text.toString().trim()

        if (amountStr.isEmpty()) {
            Toast.makeText(this, "Please enter an amount",
Toast.LENGTH_SHORT).show()
            return
        }

        val amount = amountStr.toFloatOrNull()
        if (amount == null) {
            Toast.makeText(this, "Enter a valid amount",
Toast.LENGTH_SHORT).show()
            return
        }

        if (amount < 0f) {

```

```

        Toast.makeText(this, "Amount cannot be negative",
Toast.LENGTH_SHORT).show()
        return
    }

    val dayIndex = binding.spinnerDays.selectedItemPosition
    expenses[dayIndex] = amount

    updateCharts()
    updateSummary()

    binding.etAmount.text?.clear()
    Toast.makeText(this, "Dashboard Updated", Toast.LENGTH_SHORT).show()
}

private fun setupCharts() {
    val xAxisFormatter = IndexAxisValueFormatter(labels)

    // Bar Chart Setup
    binding.barChart.apply {
        description.isEnabled = false
        setFitBars(true)
        animateY(1000)
        legend.apply {
            isEnabled = true
            textSize = 12f
            form = Legend.LegendForm.SQUARE
        }
        xAxis.apply {
            valueFormatter = xAxisFormatter
            position = XAxis.XAxisPosition.BOTTOM
            setDrawGridLines(false)
            granularity = 1f
            textSize = 12f
        }
        axisRight.isEnabled = false
        axisLeft.axisMinimum = 0f
        axisLeft.textSize = 12f
    }

    // Line Chart Setup
    binding.lineChart.apply {
        description.isEnabled = false
        animateX(1000)
        legend.apply {
            isEnabled = true
            textSize = 12f
        }
        xAxis.apply {
            valueFormatter = xAxisFormatter
            position = XAxis.XAxisPosition.BOTTOM
            setDrawGridLines(false)
            granularity = 1f
            textSize = 12f
        }
        axisRight.isEnabled = false
    }
}

```



```

        axisLeft.axisMinimum = 0f
        axisLeft.textSize = 12f
    }

    // Pie Chart Setup
    binding.pieChart.apply {
        description.isEnabled = false
        setUsePercentValues(true)
        setEntryLabelColor(Color.BLACK)
        setEntryLabelTextSize(12f)
        animateY(1400)
        centerText = "Weekly Expense %"
        setCenterTextSize(18f)
        holeRadius = 45f
        transparentCircleRadius = 50f
        legend.apply {
            isEnabled = true
            textSize = 12f
        }
    }

    updateCharts()
}

private fun updateCharts() {
    updateBarChart()
    updateLineChart()
    updatePieChart()
}

private fun updateBarChart() {
    val barEntries = ArrayList<BarEntry>()
    for (i in expenses.indices) {
        barEntries.add(BarEntry(i.toFloat(), expenses[i]))
    }

    val barDataSet = BarDataSet(barEntries, "Daily Expense").apply {
        colors = ColorTemplate.MATERIAL_COLORS.toList()
        valueTextSize = 11f
        valueTextColor = Color.BLACK
        setDrawValues(true)
    }

    val barData = BarData(barDataSet).apply {
        barWidth = 0.6f
        setValueFormatter(CurrencyValueFormatter())
    }

    binding.barChart.data = barData
    binding.barChart.invalidate()
}

private fun updateLineChart() {
    val lineEntries = ArrayList<Entry>()
    for (i in expenses.indices) {
        lineEntries.add(Entry(i.toFloat(), expenses[i]))
    }
}

```

```

    }

    val lineDataSet = LineDataSet(lineEntries, "Weekly Expense
Trend").apply {
        color = Color.BLUE
        setCircleColor(Color.BLUE)
        valueTextColor = Color.BLACK
        valueTextSize = 11f
        lineWidth = 3f
        mode = LineDataSet.Mode.CUBIC_BEZIER
        setDrawFilled(true)
        fillColor = Color.CYAN
        fillAlpha = 50
        setDrawValues(true)
    }

    val lineData = LineData(lineDataSet).apply {
        setValueFormatter(CurrencyValueFormatter())
    }

    binding.lineChart.data = lineData
    binding.lineChart.invalidate()
}

private fun updatePieChart() {
    val pieEntries = ArrayList<PieEntry>()
    for (i in expenses.indices) {
        if (expenses[i] > 0f) {
            pieEntries.add(PieEntry(expenses[i], labels[i]))
        }
    }

    if (pieEntries.isEmpty()) {
        binding.pieChart.clear()
        binding.pieChart.centerText = "No Data Available"
        binding.pieChart.invalidate()
        return
    }

    val pieDataSet = PieDataSet(pieEntries, "Expense Share").apply {
        colors = ColorTemplate.JOYFUL_COLORS.toList()
        sliceSpace = 3f
        valueTextSize = 12f
        valueTextColor = Color.BLACK
    }

    val pieData = PieData(pieDataSet)
    binding.pieChart.data = pieData
    binding.pieChart.centerText = "Weekly Expense %"
    binding.pieChart.invalidate()
}

private fun updateSummary() {
    val total = expenses.sum()
    val maxExpense = expenses.maxOrNull() ?: 0f
    val highestDayIndex = expenses.indexOf(maxExpense)
}

```

```

        val highestDay = if (highestDayIndex >= 0) labels[highestDayIndex]
else "-"

        binding.tvSummary.text = "Total Expense: ${formatCurrency(total)}"
        binding.tvHighest.text = "Highest Spending Day: $highestDay
(${{formatCurrency(maxExpense)}})"
    }

    private fun formatCurrency(value: Float): String {
        return String.format(Locale.US, "%.2f", value)
    }

    private class CurrencyValueFormatter : ValueFormatter() {
        override fun getFormattedValue(value: Float): String {
            return String.format(Locale.US, "%.0f", value)
        }
    }
}

```

Output



Result

Thus, a **Data Visualization App** was successfully designed and developed to display user data using charts and graphs, providing clear and effective graphical representation of the information.